

Rapport

Projet Intégration IA et Interaction

SRI 2025-2026



CookAssist

Alex Rhodes - Eliot Pazzé

Nelson Sanchez - Hoang Long Duong

Sommaire¹

Sommaire	1
1. Introduction générale	2
2. Problématique	3
2.1 Contexte	3
2.2 Objectifs et nature des missions à effectuer	3
3. Architecture et Choix Technologiques	4
3.1 Stack technique et stratégies de dialogue initiale	4
3.2 Interface utilisateur	5
3.3 Serveur conversationnel RASA	5
3.4 Actions personnalisées et services externes	5
4. Modélisation conversationnelle	6
4.1 Compréhension du langage naturel	6
4.1.1 Typologie des intents	6
4.2 Définition du Domaine et Gestion du Contexte	7
4.3 Scénarisation des Dialogues	8
4.3.1 Parcours utilisateur	8
5. Implémentation technique et actions Python	9
5.1 Le Serveur d'Actions RASA	9
5.2 Interface utilisateur et interaction vocale	10
6. Evaluation et tests	11
6.1 Structure des tests	11
6.2 Evaluation NLU	11
6.3 Extraction des entités	13
6.4 Gestion du dialogue	13
7. Conclusion	14
8. Annexes	16

¹ Ce rapport a été rédigé avec l'assistance de l'outil d'intelligence artificielle Gemini. Son utilisation a été strictement limitée à la reformulation stylistique de certains segments techniques (chapitre 3) ainsi qu'à la correction orthographique globale, sur la base exclusive d'un contenu technique original et de données produites par l'équipe projet. Son utilisation a été faite afin d'améliorer la clarté du document et de faciliter la lecture.

1. Introduction générale

Dans le cadre de la troisième année de SRI, le groupe d'Interaction doit concevoir un projet multimodal afin d'intégrer les compétences acquises dans différentes unités d'enseignement de la formation. Ce projet a été conçu selon un cahier des charges définissant les spécifications, les exigences et les contraintes.

Nous avons décidé de développer un assistant culinaire intelligent capable de gérer l'accompagnement des personnes en situation de handicap ou tout simplement lorsque les utilisateurs sont en train de cuisiner et ont les mains sales. Il est capable de générer des suggestions personnalisées à partir du contenu d'un réfrigérateur et de guider l'utilisateur pas à pas dans la réalisation d'un plat spécifique, en privilégiant une interaction vocale.

L'assistant culinaire contribue également à simplifier le quotidien des utilisateurs en proposant des recettes et des conseils adaptés. Les assistants classiques qui existent aujourd'hui ont été adaptés au fur et à mesure en suivant des arbres de décision rigides. Notre assistant est conçu afin de résoudre des problématiques concernant la préparation d'un plat ou suivre une recette: l'utilisateur a souvent les mains occupées ou sales, rendant l'utilisation d'un écran peu pratique ou bien pour tout simplement rendre la tâche de cuisiner plus interactive et facile à suivre.

Afin de répondre à des requêtes de l'utilisateur, l'assistant repose sur l'utilisation du framework RASA pour la gestion du dialogue et de la compréhension du langage naturel, combiné à l'API OpenAI pour la génération de recettes et la synthèse vocale. L'ensemble vise à démontrer des compétences avancées en intégration de services intelligents, en architecture logicielle et en conception conversationnelle, attendues en fin de cursus SRI 5A.

Ce rapport présente le travail réalisé sur l'assistant RASA. Il est constitué de 5 parties, regroupant dans un premier temps le contexte et la problématique, puis des explications détaillant le travail technique effectué sur le backend et le frontend, et enfin une conclusion détaillant les résultats obtenus.

2. Problématique

2.1 Contexte

Un assistant culinaire purement génératif manque de contexte structurel : il peut oublier les contraintes de l'utilisateur ou perdre le fil lors d'un guidage étape par étape. À l'inverse, un chatbot classique basé sur des règles est trop rigide pour inventer une recette à partir d'ingrédients aléatoires ou variant selon les préférences des utilisateurs.

Il existe donc un risque potentiel que l'utilisateur se retrouve avec une recette incohérente ou qu'il soit obligé de manipuler son écran avec les mains sales, faute de commande vocale efficace. Par conséquent, il a été nécessaire de concevoir et implémenter une architecture hybride combinant la gestion de dialogue de RASA avec la créativité générative d'OpenAI, tout en intégrant des dispositifs d'interaction vocale performants.

2.2 Objectifs et nature des missions à effectuer

En tenant compte de la durée limitée du projet ainsi que des contraintes imposées par les intervenants, l'objectif principal a été défini comme suit :

Développer des solutions logicielles afin de garantir que l'assistant soit fiable, utilisable en mains libres et prêt à servir de prototype pour des projets futurs.

Cet objectif a été divisé en trois missions principales :

- A. **Conception du moteur NLU et des Stories RASA** : Gérer les flux de conversation multimodaux (Mode Frigo, Mode Nom) avec efficacité face aux variations de formulation utilisateur.
- B. **Intégration de l'API OpenAI** : Implémenter une génération de contenu culinaire avec contraintes techniques pour assurer la cohérence et la fiabilité des recettes générées.
- C. **Interface utilisateur avec interaction vocale** : Développer une interface permettant une utilisation mains libres adaptée à l'environnement culinaire, intégrant captation vocale (STT) et synthèse vocale (TTS).

3. Architecture et Choix Technologiques

3.1 Stack technique et stratégies de dialogue initiale

Afin de répondre à la problématique identifiée et commencer à réaliser les tâches, nous avons décidé de structurer le projet autour de trois approches principales qui seront détaillées plus en bas :

1. **RASA** : Assure la gestion du contexte conversationnel et des règles de dialogue (le choix de RASA a été imposé par le cahier des charges et ne pouvait pas être modifié).
2. **Backend Actions ([actions.py](#))** : Centralise la logique métier et les interactions avec l'API OpenAI pour la génération de recettes et l'orchestration des services externes.
3. **Interface utilisateur (Streamlit)** : Choisie pour permettre un développement rapide tout en offrant une interface intuitive et testable pour valider les capacités du chatbot.

Segmentation des modes de fonctionnement

Pour créer un système capable de détecter les intentions de l'utilisateur dans des conditions variées, nous avons défini deux zones de travail principales pour l'assistant [*Annexe A*]:

- **Mode Frigo** (`start_fridge_mode`) : L'utilisateur liste les ingrédients dont il dispose. L'assistant les collecte via une lookup table définie dans le fichier `nlu.yml` et propose une ou plusieurs recettes adaptées.
- **Mode Nom Recette** (`start_recipe_by_name`) : L'utilisateur demande un plat spécifique via son nom. L'assistant récupère ou génère la recette correspondante et peut proposer un guidage étape par étape.

Cette distinction a permis de concevoir une première version des stories pour gérer les happy paths et les cas d'erreur de façon séparée et intelligible.

3.2 Interface utilisateur

L'interface utilisateur, développée avec Streamlit, permet :

- L'affichage des échanges textuels avec l'assistant.
- La restitution des recettes générées de manière lisible et structurée.
- Le déclenchement des réponses vocales par synthèse TTS.
- L'enregistrement audio pour la capture vocale (STT).

Ce choix a favorisé un développement rapide tout en garantissant une interface conviviale (user-friendly) pour tester les capacités du chatbot et itérer rapidement sur les scénarios d'utilisation.

3.3 Serveur conversationnel RASA

Le serveur RASA a servi de composant central du système. Il est responsable de :

- La reconnaissance des intentions de l'utilisateur (intent classification).
- La gestion du contexte conversationnel via les slots.
- Le pilotage du dialogue à travers les stories et les rules.
- L'appel aux actions personnalisées pour exécuter la logique métier.

3.4 Actions personnalisées et services externes

Les actions personnalisées, codées en Python, gèrent la logique métier spécifique :

- Génération de recettes via l'API OpenAI.
- Lecture étape par étape du guidage culinaire.
- Synthèse vocale (TTS) et reconnaissance vocale (STT).

Ces actions interagissent avec les services d'OpenAI pour la création de contenu et la voix, tout en maintenant une structure fiable côté serveur et une gestion d'erreurs robuste.

4. Modélisation conversationnelle

La modélisation conversationnelle constitue l'architecture cognitive de l'assistant. Elle repose sur le RASA et est basée sur trois composantes: la compréhension des intentions (NLU), la définition de l'environnement de dialogue (Domain) et la scénarisation des interactions (Stories). L'objectif était de concevoir un dialogue fluide, naturel et suffisamment robuste pour supporter une utilisation "mains libres" en cuisine, où l'utilisateur ne peut pas constamment interagir avec l'écran.

4.1 Compréhension du langage naturel

La compréhension du langage naturel a pour rôle de transformer une phrase brute en une intention structurée compréhensible par la machine. Nous avons défini un ensemble d'intents accompagnés d'exemples d'entraînement variés pour couvrir la diversité des formulations que l'utilisateur peut exprimer dans notre contexte.

4.1.1 Typologie des intents

Nous avons structuré les intents en plusieurs catégories logiques:

- **Intents conversationnels génériques** : `greet`, `goodbye`, `affirm` et `deny` gèrent les interactions sociales de base et les validations par « oui/non ». Ces intents forment la base de n'importe quelle architecture conversationnelle.
- **Intents de démarrage** : Nous avons implémenté deux intents pour segmenter les cas d'usage : `start_fridge_mode` et `start_recipe_by_name`.
- **Intents de collecte de données** : L'intent `add_ingredient` permet une saisie en liste (ex. « J'ai des œufs et de la farine »), tandis que `done_ingredients` agit comme un marqueur de fin de séquence.
- **Intents de navigation et contrôle** : Pour le guidage pas-à-pas, les commandes `start_step_by_step`, `next_step` et `repeat_step` donnent à l'utilisateur un contrôle sur le rythme du guidage.
- **Intents d'assistance avancée** : Des commandes comme `slow_down` ou `ask_definition` enrichissent l'expérience (implémentées partiellement, à développer ultérieurement).

4.2 Définition du Domaine et Gestion du Contexte

Le fichier `domain.yml` définit l'assistant : ce qu'il peut comprendre (intents), ce qu'il peut retenir (slots) et ce qu'il peut faire (actions).

La capacité d'un assistant culinaire à répondre correctement aux requêtes repose sur sa capacité à comprendre des concepts spécifiques et à maintenir un contexte cohérent tout au long de l'interaction, interaction qui peut s'étendre sur plusieurs minutes. Pour créer et consolider cette capacité, nous avons configuré deux entités principales :

- **ingredient** : Extrait les ingrédients directement depuis la requête de l'utilisateur.
- **recipe_name** : Extrait les noms de plats demandés par l'utilisateur.

Gestion des slots

Une fois les ingrédients ou le nom de la recette récupérés, ils sont stockés à court terme grâce aux slots pour assurer la continuité conversationnelle. La gestion mémoire est structurée selon plusieurs axes :

- **Contexte de la recette** : Stockage de la recette globale
- **Structure de données générée** (`recipe_json`) : Recette au format JSON générée par OpenAI
- **Nom du repas souhaité** : Identification du plat en cours
- **Gestion d'état du guidage** : Pilotée par le curseur `step_index` et sécurisée par une mémoire tampon `last_step_text` pour les répétitions sans recalcul.
- **Préférences utilisateur** : Gestion du slot `speech_rate` adapte la vitesse de synthèse vocale.

Continuité conversationnelle

La fluidité de l'interaction est garantie par l'utilisation de réponses prédéfinies (« `utter_response` »). Ces modules permettent d'assurer une cohérence discursive en fournissant des consignes claires et concises à l'utilisateur, tout en établissant des transitions fiables entre les différentes phases du scénario.

Gestion des objets complexes

À cette étape, nous avons identifié et résolu des problèmes liés à l'utilisation de RASA pour le stockage d'objets complexes :

RASA est conçu pour gérer des états conversationnels simples, mais pas pour stocker directement des objets complexes comme une recette entière structurée (JSON) générée par un LLM externe.

Solution implémentée :

Nous avons utilisé les slots de type any (slot recipe_json) pour stocker la recette générée via appel API sans que RASA n'essaie de l'interpréter pour la prédiction du dialogue.

Nous avons également utilisé des slots de type float (pour step_index, nb_personnes et temps_max) pour gérer la logique de progression de manière déterministe, séparant ainsi le contenu de la recette de la logique de navigation étape par étape et les paramètres.

4.3 Scénarisation des Dialogues

Les Stories sont des exemples de dialogues qui entraînent le modèle probabiliste de RASA à prédire la prochaine action. Elles relient les intents aux actions.

4.3.1 Parcours utilisateur

Nous avons modélisé plusieurs scénarios types pour couvrir les cas d'usage nominaux et alternatifs :

- **Scénario “Frigo”** : L'utilisateur dit bonjour puis donne une série d'ingrédients, reçoit une suggestion de recette et valide, puis bascule en mode guidage pas-à-pas optionnel.
- **Scénario “Expert” (Direct)** : L'utilisateur donne directement ses ingrédients sans phrase d'introduction (ex. « J'ai des tomates »). L'assistant doit comprendre le contexte implicite et lancer la recherche de recettes automatiquement.
- **Scénario “Recette par nom”** : Demande explicite d'une recette spécifique suivie d'un choix entre lecture globale ou guidée étape par étape.
- **Scénario de Gestion d'Interruptions** : Intègre les demandes de pause, de répétition ou de définition au milieu d'un guidage (implémentation non terminée), sans perdre le fil de l'étape en cours.

Ces scénarios ont été pensés de manière à combler une difficulté que l'on peut retrouver lorsqu'on cuisine avec un assistant : en cuisine, l'utilisateur suit rarement un chemin parfaitement linéaire. Il peut demander à répéter trois fois, puis demander une définition, puis faire une pause. Si le modèle est trop rigide, basé uniquement sur des règles, il échoue. S'il est trop souple, il perd le contexte. Il était donc nécessaire de trouver une solution hybride combinant rigidité et flexibilité.

Gestion des transitions critiques

Les transitions critiques, comme passer de la validation de la recette au démarrage du guidage, sont gérées par des **rules** pour garantir la sécurité du flux.

Pour ce qui est de la navigation interne entre étapes, celle-ci est gérée par les **stories** avec augmentation de données et redondance partielle, afin d'apprendre à RASA que les commandes de navigation (`next_step` et `repeat_step`) sont valides à tout moment du guidage, peu importe l'historique récent.

5. Implémentation technique et actions Python

Ce chapitre détaille la réalisation technique du projet, depuis la logique métier hébergée sur le serveur d'actions RASA jusqu'à l'interface utilisateur vocale développée sous Streamlit. Dans cette partie nous allons décrire les différents choix d'architecture nécessaires afin de créer un moteur conversationnel déterministe grâce à l'aide de l'IA générative.

5.1 Le Serveur d'Actions RASA

L'Action server est le composant chargé d'exécuter la logique qui ne peut être gérée par les règles de dialogue. Dans ce projet, il assure le pont entre les intentions de l'utilisateur et les services d'OpenAI. Il gère :

- La collecte des ingrédients.
- La génération de recettes.
- Le guidage étape par étape.
- La synthèse vocale (TTS) La reconnaissance vocale (STT).

Nous avons structuré le fichier `actions.py` de manière modulaire, agissant comme point d'entrée principal pour RASA en important et exposant les classes définies dans d'autres fichiers Python (`recipe_actions.py`, `schemas.py`, etc).

L'intégration des modèles de langage dans une architecture logicielle qui utilise RASA pose un problème d'exploitation de données textuelles non structurées; Rasa utilise une gestion d'états via slots typés nous obligeant à modifier la sortie du LLM en JSON strict pour que ses réponses puissent être lues et traitées par le code Python.

Rôle de `schemas.py` : Ce fichier contraint le modèle d'OpenAI à structurer les réponses de manière précise et prévisible. La liste des ingrédients ainsi que les étapes ordonnées sont mises en format JSON, garantissant un traitement déterministe et une fiabilité lors de l'affichage et de la synthèse vocale.

L'architecture de données est exploitée au sein du module `recipe_actions.py` pour implémenter les deux modes de fonctionnement du robot : « frigo » et « nom ». L'interaction et sa continuité sont assurées par l'action `action_tell_recipe_step` qui agit comme un itérateur intelligent basé sur un index de progression (`step_index`). Cet itérateur permet de séquencer la lecture de la recette préalablement stockée et offre à l'utilisateur un contrôle sur le rythme du guidage vocal sans nécessiter un recalcul par le modèle génératif.

5.2 Interface utilisateur et interaction vocale

L'interface utilisateur, développée avec Streamlit, a été conçue pour répondre à une contrainte majeure du projet : permettre une utilisation mains libres adaptée pour la cuisine. Afin de mettre en œuvre cette fonctionnalité, nous avons dû résoudre certains problèmes liés au mode de fonctionnement événementiel des navigateurs web et au cycle de rafraîchissement de Streamlit.

Concernant la capture audio (STT), les premiers tests basés sur une détection automatique de la voix ont été difficiles à mettre en œuvre dans un environnement web standard; Le maintien d'un flux audio ouvert nécessiterait une gestion avancée de WebSockets parallèles via des bibliothèques pour contourner le modèle requête-réponse standard envisagé dès le début.

Afin de résoudre ce problème, nous avons opté de faire une approche "Push-to-Talk" activée par la barre d'espace et de cette manière garantissant un contrôle utilisateur précis. Ceci dit, c'est une première approche faite afin de tester le prototype et cela reste à améliorer dans le futur afin que notre prototype soit 100% mains libres.

Quant à la restitution vocale (TTS), elle repose sur une gestion entre le serveur d'action et l'interface client. Côté serveur `action_text_to_speech` assure la génération du fichier audio via l'API OpenAI TTS et sa sauvegarde locale. Le rafraîchissement systématique de la page par Streamlit à chaque interaction utilisateur posait des problèmes de redémarrage en boucle ou de coupures du flux audio. Afin de résoudre cela, nous avons implémenté une gestion d'état en lien avec une mécanique de hachage audio[Anexe B]. Cela nous permet de vérifier si un fichier a déjà été lu, garantissant la fluidité conversationnelle et que cette fluidité puisse paraître "naturelle" où la réponse de l'assistant est donnée une seule fois sans interruption.

Concernant la configuration du projet par rapport à la gestion des API et les clefs, celles-ci sont externalisées via des variables d'environnement telles que OPENAI_API_KEY, OPENAI_MODEL ou TTS_OUTPUT_DIR. Nous avons également intégré des mécanismes de sécurité et de fallback au sein du système pour donner un niveau de robustesse. Ainsi, en cas de défaillance de l'API de génération, le système bascule vers un mode de compatibilité, et des messages d'erreur contrôlés ont été prévus pour éviter tout arrêt critique de l'application. Ceci permet à l'utilisateur de ne pas se retrouver face à un dysfonctionnement technique non géré.

6. Evaluation et tests

Cette section présente le protocole et les résultats des tests servant à évaluer la fiabilité globale du système, notamment la robustesse du NLU et la cohérence des dialogues.

6.1 Structure des tests

Le protocole de test est divisé en trois niveaux :

1. **Tests univariés NLU** : Évaluation de la capacité du modèle à extraire correctement les intentions et les entités, avec métriques de précision, recall et F1-score.
2. **Tests end-to-end** : Vérification des parcours conversationnels complets avec 13 scénarios de test couvrant les cas nominaux et les cas d'erreur.
3. **Analyse des matrices de confusion** : Identification des zones d'ambiguïté du modèle pour amélioration future.

6.2 Evaluation NLU

En analysant le rapport de classification intent_report.json, nous pouvons observer que celui-ci révèle une performance globale excellente du modèle DIETClassifier (composant d'architecture de réseau de neurones utilisé par RASA) avec une précision moyenne de 99.5% et un score F1 de 0.99% (sur 493 exemples de test) [Annexe C] :

```
"weighted avg": {  
  "precision": 0.998090919937955,  
  "recall": 0.9979716024340771,  
  "f1-score": 0.9979824494798841,  
  "support": 493  
}
```

Tous les intents atteignent une précision de 1.0, garantissant une compréhension fluide dans les scénarios visés. Seuls les intents `affirm` et `next_step` présentent une légère confusion (observée dans 1 cas : « vas-y ») :

```
intent_errors.json :  
[  
  {  
    "text": "vas-y",  
    "intent": "affirm",  
    "intent_prediction": {  
      "name": "next_step",  
      "confidence": 0.9752892851829529  
    }  
  }  
]
```

Analyse : Cette confusion peut être expliquée par le fait que certaines expressions courtes (« vas-y ») peuvent être interprétées comme une affirmation ou comme une commande pour continuer. Cela reste marginal puisque la confiance du modèle est de 97%, indiquant une très haute certitude malgré la mauvaise classification dans ce cas limite.

6.3 Extraction des entités

D'après le `DIETClassifier_report` [Annexe D] on obtient une précision de **99.09%** pour l'entité `liste_ingredients` sur 329 exemples de test.

À travers l'analyse des erreurs (`DIETClassifier_errors`), nous avons observé quelques cas limites :

Dans la phrase "je veux faire une carbonara" ou ""montre-moi la recette de soupe à la tomate", le modèle détecte correctement "carbonara" et "soupe à la tomate" comme `nom_recette` avec une confiance de 60%, ce qui est suffisant pour déclencher l'action mais indique une marge de progression sur les recettes moins courantes (comme comparé à des recettes de lasagnes avec 88% ou crêpes avec 77%)

Sur "j'ai des tomates et des oignons", le modèle sépare parfaitement les deux entités `liste_ingredients` avec des scores de confiance élevés (79% et 59%), validant notre approche de collecte d'ingrédients multiples.

6.4 Gestion du dialogue

La gestion du dialogue a été validée par l'exécution de 13 stories de test simulant des conversations réelles entre l'utilisateur et l'assistant :

- Mode frigo
- Mode nom
- Suivi d'une recette pas à pas
- Guidage
- Recette refusée
- Recette par nom refusée
- Utilisateur pas prêt
- Utilisateur demande de répéter
- Utilisateur demande passage à l'étape suivante
- (et autres scénarios).

Le rapport de test `story_report.json` et la matrice de confusion des stories `story_confusion_matrix.png` [Annexe E] confirme un taux de réussite de 100% sur l'ensemble des conversations de test, indiquant que tous les scénarios se déroulent sans aucune déviation du chemin attendu.

7. Conclusion

Ce projet a permis de développer un assistant culinaire intelligent combinant avec succès la gestion de dialogue de RASA, la puissance générative d'un modèle simple d'OpenAI ainsi qu'une interface graphique et vocale semi-mains libres.

Le développement RASA a pris du temps du fait d'une décision initiale d'utiliser la version RASA 3 la version la plus récente. Cependant, cette version a besoin de dépendances très spécifiques, non fournies lors de l'installation et payantes pour être efficace. Face à ce problème et en tenant compte de l'expérience acquise lors des travaux pratiques d'agents conversationnels utilisant RASA 2.8, nous avons pris la décision (assez tardive dans le projet) de revenir à cette version bien maîtrisée par tous. Cela a entraîné des problèmes de compatibilité avec l'ancienne version de Python 3.8 que RASA 2.8 utilise et nos autres dépendances qui nécessitent des versions plus récentes.

Même si cette décision est arrivée assez tard, cela nous a permis d'accélérer le développement en fin de projet.

Résultats atteints :

- NLU avec **99.8%** de précision sur la classification des intents.
- Extraction d'entités à **99.09%** de précision.
- **100%** de taux de réussite sur les 13 scénarios de test end-to-end.
- Architecture modulaire et extensible.
- Prototype quasi-fonctionnel prêt pour des développements futurs.

Bien que le prototype actuel démontre la viabilité d'une architecture hybride RASA-LLM pour le développement d'un assistant culinaire, plusieurs axes d'évolution ont été identifiés pour transformer ce prototype en un produit pleinement opérationnel et ergonomique.

L'amélioration prioritaire concerne l'affranchissement du mode push-to-talk. L'implémentation d'une détection vocale continue (VAD - Voice Activity Detection) permettrait une interaction réellement "mains libres", essentielle à notre contexte de départ. Cela supprimerait la nécessité d'interagir physiquement avec le clavier, rapprochant l'expérience utilisateur de celle des enceintes connectées.

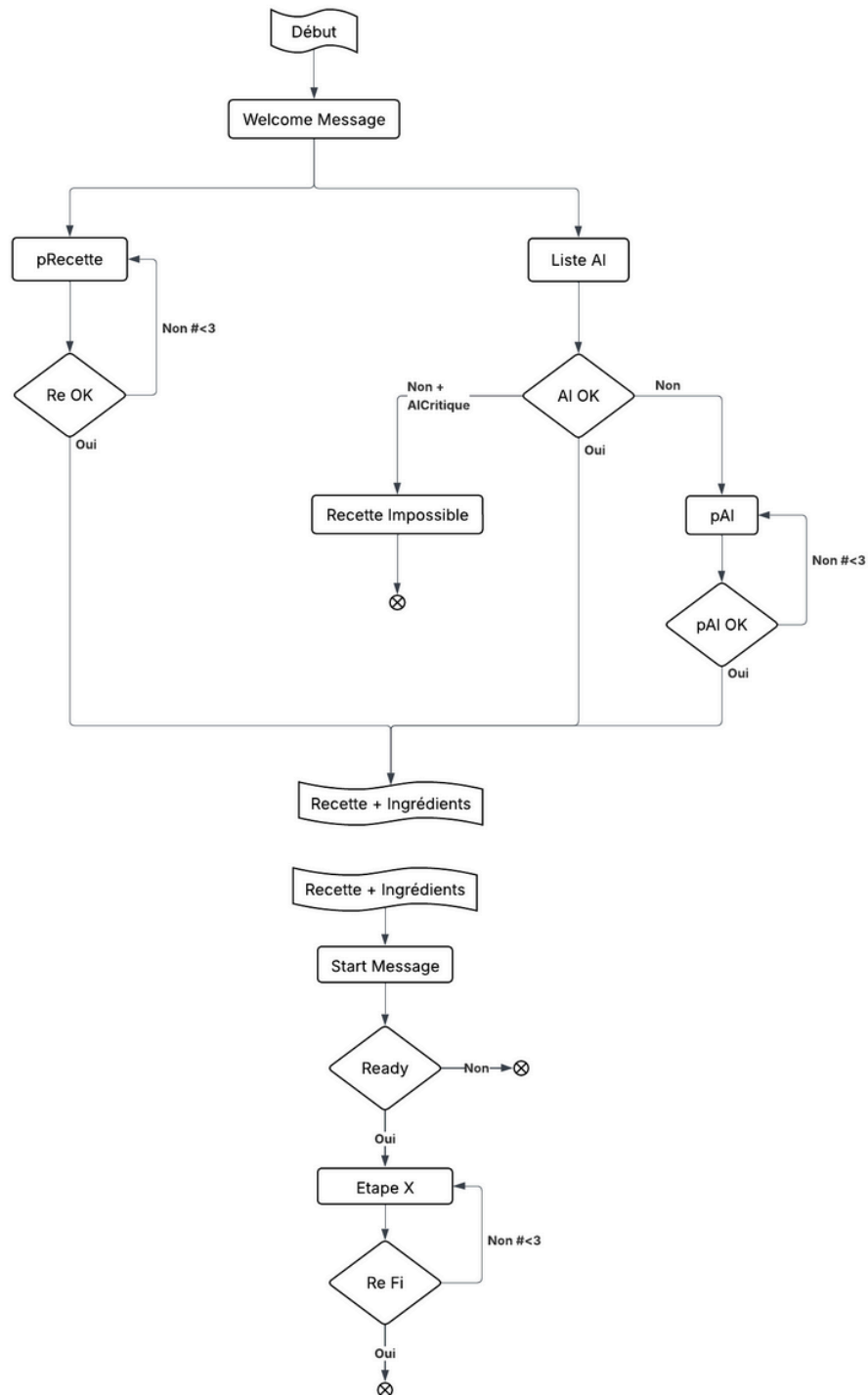
Concernant ambiguïté entre les intentions affirm et next_step sur des énoncés courts (ex: "vas-y"): Un enrichissement du dataset d'entraînement et l'ajustement des hyperparamètres du DIETClassifier peuvent être faits pour corriger ces cas limites. Par ailleurs, il nous manque encore les actions contextuelles, telles que la mise en pause du guidage ou la demande de définitions culinaires.

La latence étant critique pour une interface vocale fluide, une optimisation du nombre de requêtes vers l'API OpenAI est envisageable, potentiellement par la mise en cache de

réponses fréquentes ou l'utilisation de modèles plus légers pour les tâches simples. Cela réduirait simultanément les coûts d'exploitation et le temps de réponse perçu par l'utilisateur.

Enfin, le projet gagnerait à être étendu à une audience plus large. Cela passe d'abord par l'internationalisation du modèle, ce qui a déjà été abordé lors du développement (support de l'anglais et de l'espagnol par exemple), et ensuite par un déploiement sur des supports matériels dédiés. Le portage de l'assistant sous forme de Skill pour enceintes intelligentes (Google Home ou Alexa) ou de bot sur application conversationnelles (Discord ou WhatsApp) constituerait l'aboutissement logique de ce projet d'intégration.

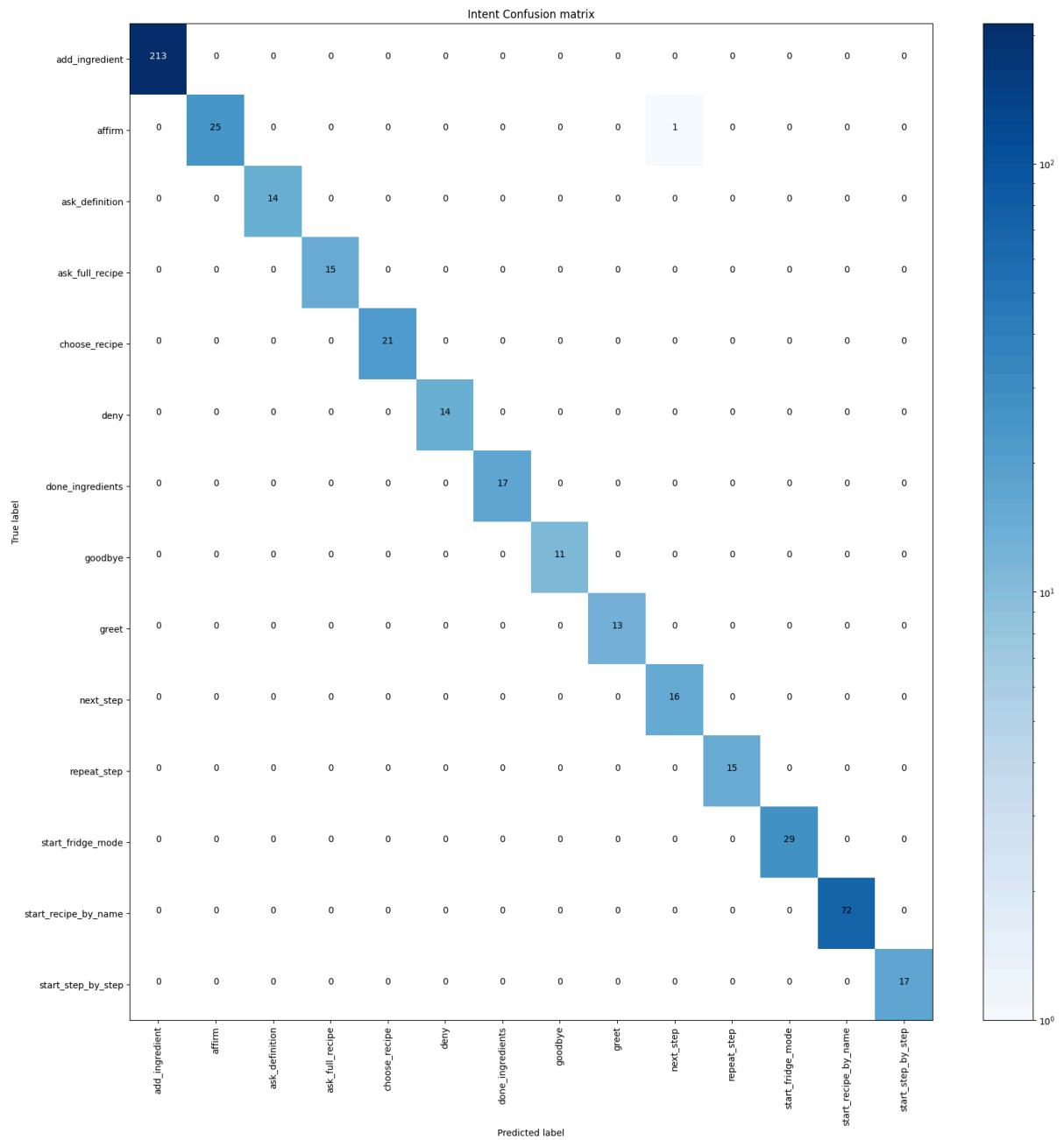
8. Annexes



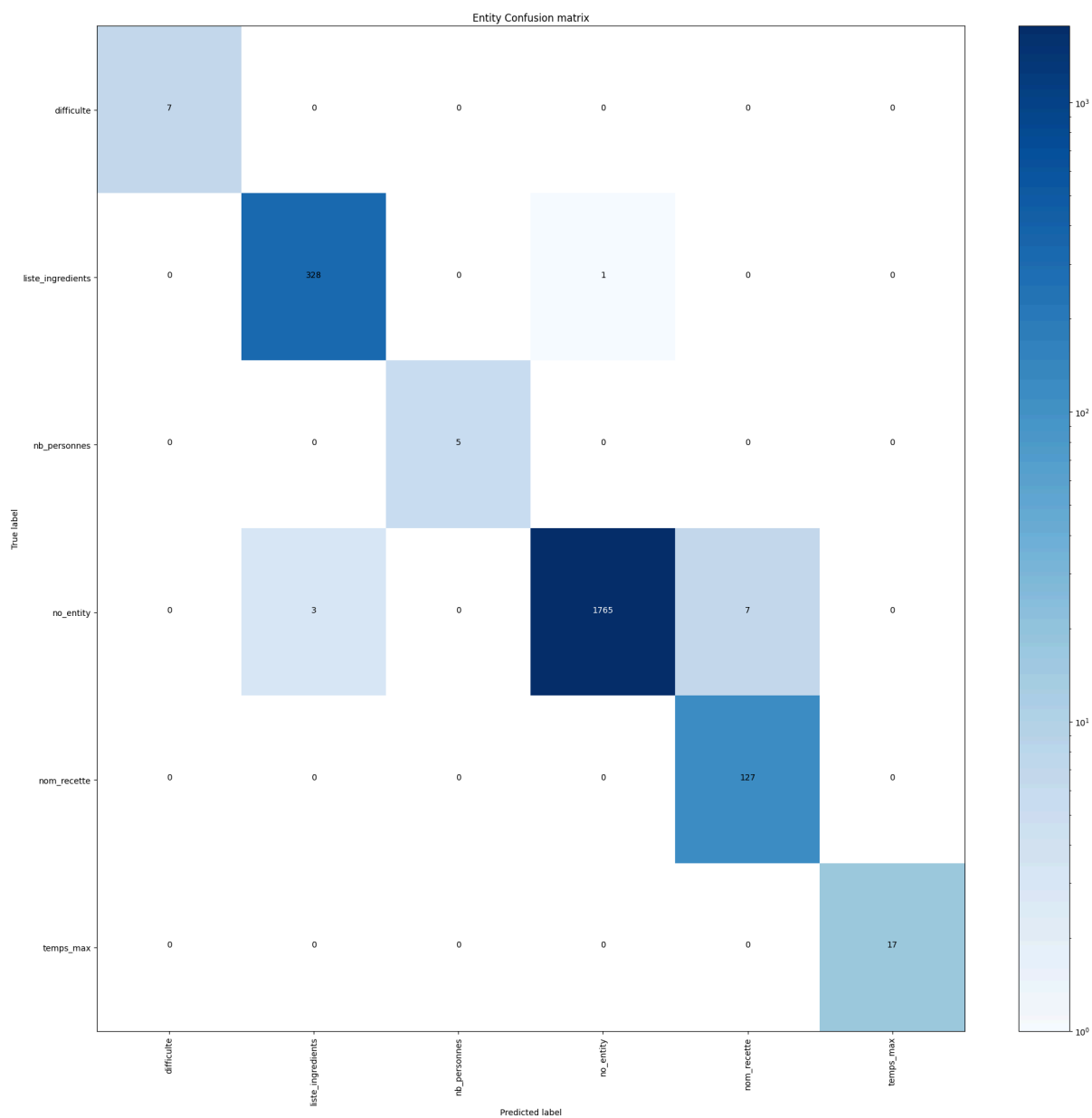
Annexe A : Diagramme conversationnel fait la première journée

Nous pouvons créer la fonction `_render_local_audio_file` de [streamlit-app.py](#) en utilisant `hashlib` (<https://docs.python.org/3/library/hashlib.html>) pour générer un identifiant unique et stable pour le lecteur audio HTML. Cela empêche Streamlit de recharger le composant lors des rafraîchissements de l'interface

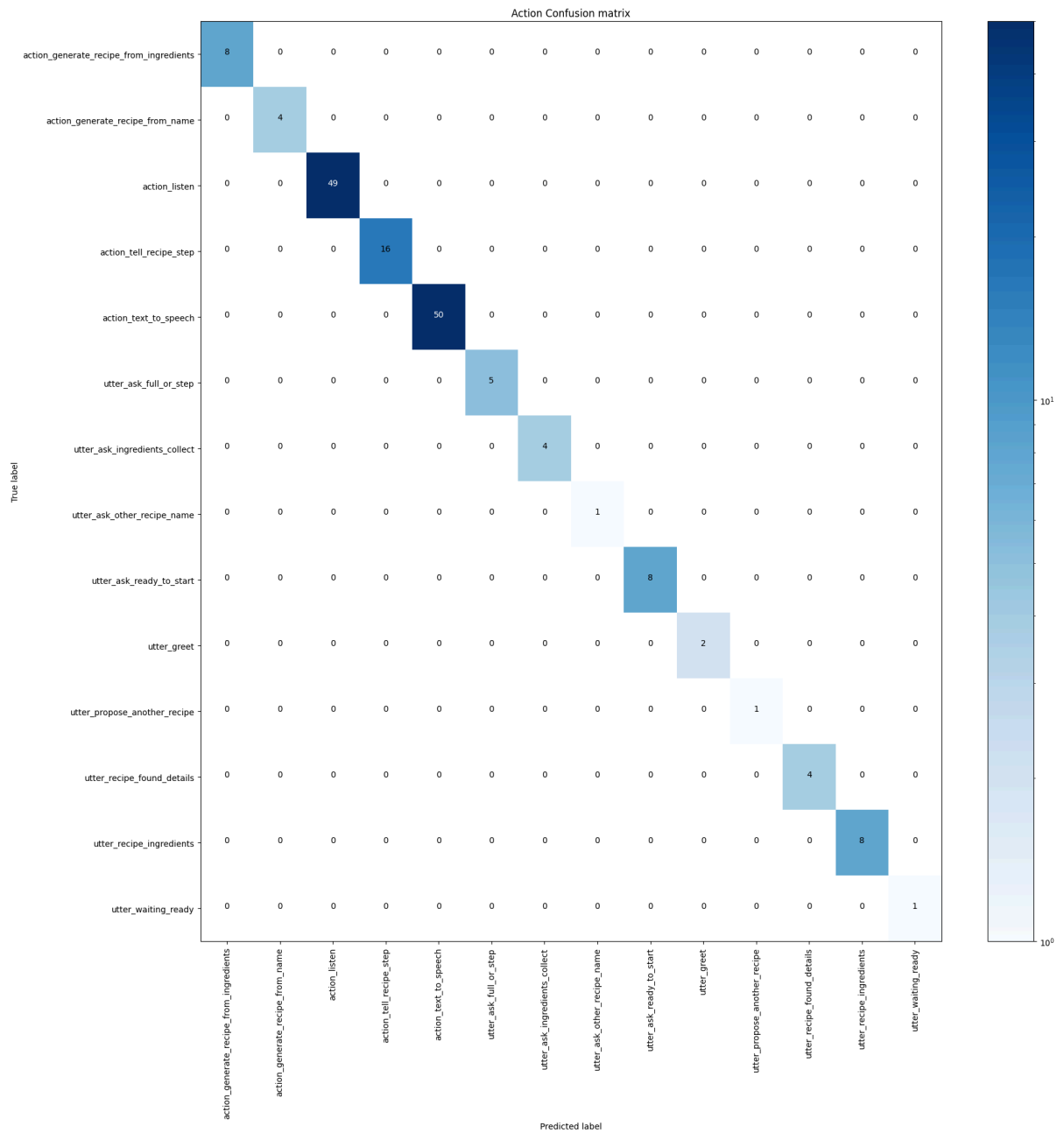
Annexe B : Explication concernant le hachage



Annexe C : Matrice de confusion presque parfaite des intents, avec les labels et les textes predicts. Observez le cas d'erreur (1 seul) entre affirm et next_step.



Annexe D : Matrice de confusion des labels



Annexe E : Matrice de confusion des stories.